

Unit A — Structured Intelligence: Status Report

1. Current Status & Decisions Made

Scope and shape

- Unit A is request-driven and synchronous, behind the existing gate chain. No background work, no scheduler, no new persistence layer.
- Two paths in scope: note classification with the clarification loop, and quality scoring. The daily brief is deferred pending separate scoping.
- Design A is settled: the note saves to the CRM first, we read it, judge it, and update it afterwards. Forced by the requirement that a scoring outage must never block a save.
- The backend stores all results. We hold no note text at rest.

Business rules

- Every note is classified into one of six types first. The bar applied depends on the type; no-contact notes are exempt from the full checklist.
- A note is vague if it misses any of: what happened, what the client said or wanted, or a next step with a date.
- Scoring weights total 100: what happened 25, what the client said 20, next step and date 25, deal specifics 20, clarity 10.
- Scores are shown as bands, not numbers: Poor, Fair, Good, Excellent.
- The rep is prompted once only, then the note stores as written. The prompt must be specific, not generic.
- Maximum three prompts per person per hour.
- Enforcement is advisory at launch. Blocking is built but shipped switched off.
- Arabic, English, and mixed notes are judged identically.
- Scores are for coaching only, never pay, commission, or discipline. Original notes are never deleted or replaced. Nothing is ever invented to fill a field.

Technical decisions

- Unit A owns the `core/llm/` seam; Unit B consumes it.
- Request counters and token counters stay separate, never merged or converted.
- LLM calls use `retry=False` and an explicit longer timeout passed per call, never a raised global.
- Malformed output: validate, reprompt once with a stricter instruction, then an enumerated error. The bad output is never fed back into the prompt.
- Idempotency key: tenant, operation, lead id, content fingerprint. Tenant is mandatory.
- A third Redis client holds operational state: the attempt counter, the rate limit, and the idempotency seen-keys.

- Every score records its rubric, prompt, and model version. Past notes are never re-scored when rules change.
- Unit-owned schemas live in the unit, not in root [schemas/](#).
- Workers fail closed on a counter failure; the request path stays fail-open. A deliberate third policy.

2. Pending Decisions

- The shape of the administrator-changeable rules record: where weights, thresholds, enforcement mode, and note-type coverage live at runtime, and whether they are per tenant. Needed before scoring is built.
- Where blocking enforcement attaches. It is described as applying at stage changes, but that integration point does not exist in the pipeline and has no endpoint.
- Whether the model returns component marks only, with the total and band computed in code. Strongly indicated but not formally settled.
- Model selection, across three separate tasks: classification, vague detection, and scoring. Drives the cost-per-thousand-notes answer owed to the business.
- The [/ready](#) body detail when the operational store is degraded.
- Two open codebase facts: what the prompt builder returns, and whether the cost Lua script takes key names as parameters.
- Housekeeping still to be recorded: several settled decisions are not yet in ASSUMPTIONS.

3. Blockers & Risks

Hard blocks

- No backend write endpoint. Design A means the ask is an update on an existing note, including whether it accepts an idempotency key header. Nothing about the write-back can be built until this lands.
- Per-rep aggregation depends on the backend exposing stored scores. The ask should be raw scores with a date filter, not a pre-computed aggregate.

Sequencing blocks

- The real model cannot be wired until the tool layer makes one successful live call to the backend. Phase 0 exit criterion 3 is still open.
- Evaluation cannot start until the note sample and the hand-scored calibration set arrive.

Risks

- Within-tenant exposure: any rep can read any lead's notes. The service credential returns the whole tenant and Gate 3 is parked. Accepted pilot risk, but it must stay written down.

- Configurability retrofit: if rules are read from startup config and later rebuilt as a runtime record, the scoring path, banding, schemas, and tests all get rewritten. Cheap now, expensive later.
- Acceptance targets are optimistic for a first release, particularly ninety percent flagging accuracy and eighty-five percent band agreement across three language conditions. They should be reframed as calibration targets with adjacent-band agreement counted as agreement.
- Cost overshoot: a single request can make several LLM calls and pass the cap partway through. Mitigated by a pre-flight check, not eliminated.
- Unknown whether adding a note bumps the parent lead's **updatedAt**. If it does not, any date-filtered bulk read silently misses notes on older leads. A wrong-answer bug, not a crash.
- Design A leaves a weak note in the CRM until it is improved. An accepted trade-off for availability.

4. Key Considerations

Boundaries that must not move

- The note is untrusted text and the model's output is untrusted too. Both are contained structurally, not by filtering.
- Prompts are assembled server-side only. The model client passes the prompt through unchanged, or the injection boundary leaves the one place it is tested.
- Every model output is schema-validated before anything downstream touches it. Fails closed.
- Client responses stay generic and come from a fixed set of messages with no interpolation. Real reasons go only to the audit log. Raw note text and model output are never logged.

Rules that shape the design

- Fail-open and fail-closed are deliberate and differ by dependency. Auth and tenancy deny. Cost allows. Workers deny. An unavailable idempotency check cannot be read as no duplicate, so it denies too.
- Requests and tokens are different units. Mixing them makes both limits meaningless.
- Anything the business changes without a release is configuration, not code. Weights and thresholds are configuration; the pipeline is code.
- Prompts are code: versioned, stable prefix first so provider caching works, and the version stored with every output.
- Reads and writes have different retry policies. A paid call that may have already completed is treated like a write.

Edge cases worth holding

- A good note needs no prompt, so it must not consume the hourly allowance. Check the limit early, increment only when a prompt is actually sent.
- A resubmit is a new request and the service remembers nothing, so the attempt count must persist outside the request.

- A retry that is byte-identical to the first attempt returns the same answer. The stricter instruction must differ, and must sit in the variable tail so the cached prefix still hits.
- Truncated output and confused output are different failures. The finish reason distinguishes them.
- Lead ids collide across tenants, since each tenant has a separate database. Every key includes the tenant.
- Several things are keyed on a content fingerprint in the same store. They need clearly distinct namespaces.
- Models are unreliable at arithmetic, and the score is a weighted sum. That argues for computing the total in code.